

2025 오픈소스프로그래밍 프로젝트 최종보고서

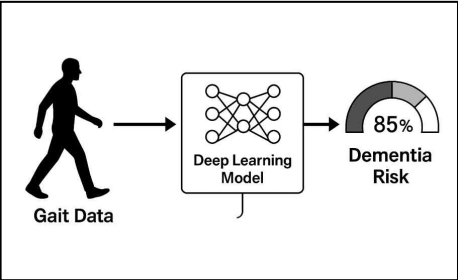
프로젝트명	-			
팀명		팀장	학번	이름
수행기간	2025. 05. 15 ~ 2025. 06. 22 (학기말까지)	팀원		

나. 프로젝트 개발 내용

1. 연구개발 내용

본 연구개발에서는 [그림 1]과 같이 걸음걸이 데이터를 기반으로 치매 고위험군을 예측하는 딥러닝 모델을 개발하고자 하며, 이를 위해 다음 두 모듈을 주요 설계 기준으로 정한다.

- Gait Data Preprocessing 모듈 (걸음걸이 데이터 전처리 모듈)
- Gait Dementia Risk Prediction 모듈 (치매 고위험군 예측 모듈)



[그림 1. 걸음걸이 기반 치매 고위험군 예측 시스템]

2. Gait Data Preprocessing 모듈 (걸음걸이 데이터 전처리 및 특징 추정 모듈)

(1) 걸음걸이 데이터 구조 및 특성 분석

본 프로젝트에서는 [그림 2]와 같이 시간에 따른 보행자 움직임 데이터를 CSV 파일 형식으로 수집함. 이 데이터는 활동 관련 데이터, 시간 관련 데이터, 활동 강도별 시간 데이터 등 다양한 수치 데이터가 포함되어 있음.

(2) 데이터 전처리 및 정규화 처리

- 결측치 및 이상치 제거

데이터의 결측치 및 이상치를 제거하여 데이터를 정제함. 이상치는 IQR 기준으로 탐지하고 히스토그램을 시각화하여 이상치 제거 여부를 확인함

- 정규화(Z-score Standization)

StandardScaler를 이용하여 모든 수치형 feature에 대해 평균 0, 표준편차 1로 정규화함으로써 학습을 안정화시키고 비교가 가능하도록 만듦.

- KMeans 클러스팅

KMeans 클러스팅을 적용하여 유사한 걸음걸이 패턴을 갖는 그룹을 나눔. 이로써 군집별 활동 수준의 평균을 비교하고 특정 활동 패턴을 가진 그룹을 치매 고위험군의 가능성이 있다고 평가할 수 있음.

- Silhouette Score을 통한 클러스터 수 검증

KMeans의 클러스터 수 k를 2~10까지 변화시키며 Silhouette Score를 계산해 군집 품질을 수치적으로 평가함. 이를 통해 최적의 k값 선택함.

(3) 특징 추출 및 입력 벡터 구성

- 최종 feature 구성

모델 학습을 위해 다음 8개 칼럼을 입력 feature로 선택함.

'activity_score', 'activity_steps', 'activity_inactive', 'activity_high',
'activity_score_meet_daily_targets', 'activity_score_move_every_hour', 'activity_daily_movement',
'activity_total'

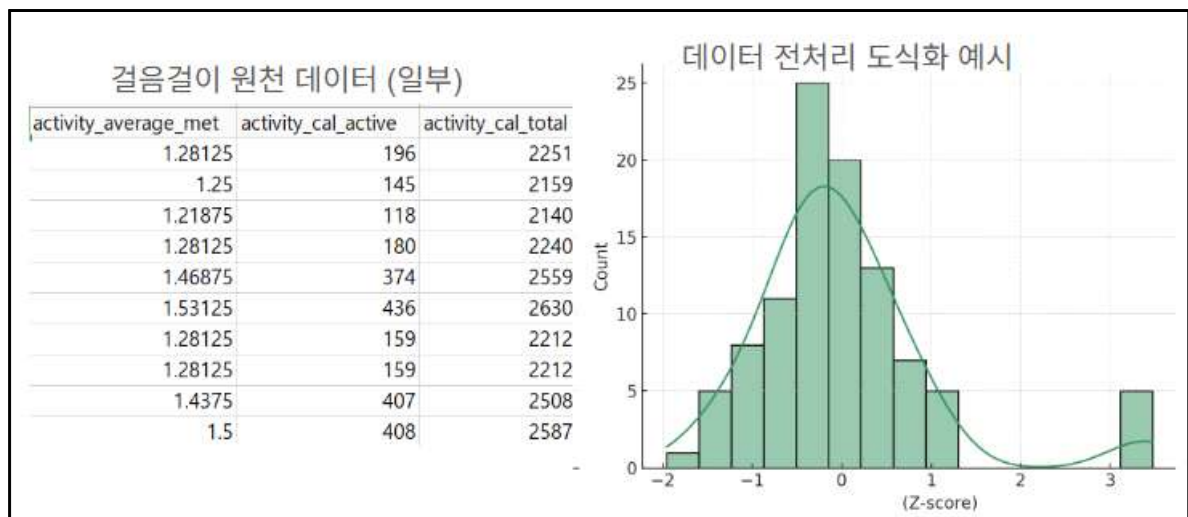
- XGBoost 분류 모델 및 SHAP 해석 적용

SHAP 기반 위험도 점수를 0~100%로 환산한 후 아래의 기준에 따라 위험도 등급을 나눔.

0~40%: 치매 저위험군, 40~70%: 중하 위험군, 70~80%: 중상 위험군, 80~100%: 고위험군

- 딥러닝 학습용 데이터 구성

최종적으로 feature(X)와 등급(Y)을 이용하여 학습/검증/테스트 셋으로 나눔. 이는 향후 딥러닝에 사용될 것임.



[그림2. 걸음걸이 데이터 예시]

3. 딥러닝 모델 설계 및 구현

(1) 딥러닝 모델 설계 분석

본 프로젝트에서는 웨어러블 기기로부터 얻은 활동 데이터를 기반으로, 고위험군의 신체 기능 저하 정도를 0~3단계 위험도로 분류하고자 하였다. 이를 위해 PyTorch 기반의 MLP 모델을 설계하여 예측 성능을 검증하였다.

(2) 데이터 전처리 및 정규화 처리

학습에 사용할 데이터는 z-score 정규화, shap를 통한 risk level별 라벨링된 8가지 활동 feature로 구성되었으며, stratify=y를 통해 라벨 비율을 유지한 상태로 학습/검증/테스트셋을 계층적 분할하였다.

20%는 테스트셋, 나머지 80% 중 10%를 검증셋으로 분리하여 모델의 일반화 성능을 모니터링할 수 있도록 설계하였다.

(3) 초기모델 설계 및 학습 세팅

- 모델 구조

[그림 3]의 모델에서 입력층에서는 총 8개의 feature를 입력받도록 설정, 은닉층은 64개의 노

드에서 시작해 32개로 점진적으로 축소되는 구조로 구성되었다. 각 은닉층에는 ReLU 활성화 함수를 적용하여 비선형성을 확보하였다. 출력층은 총 4개의 노드로 구성하여 각 클래스(0~3)에 해당하는 위험수준 확률을 산출한다. 초기 모델 구조는 과적합을 방지하고 모델의 일반적인 경향을 확인하기 위해 간단하게 설계됐으며, 이에 Dropout은 과적합 여부를 확인 후 도입 예정이다.

```
self.net = nn.Sequential(
    nn.Linear(8, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 4),
)
```

[그림3. 초기모델 구조]

- 하이퍼파라미터

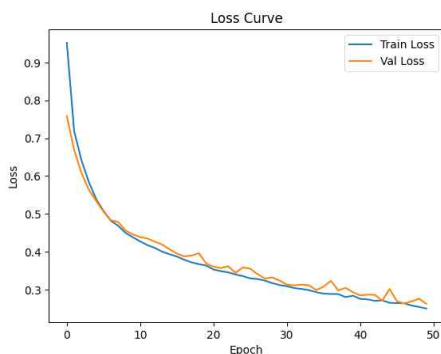
학습률은 0.001로 설정하였으며, 이는 Adam 옵티마이저의 권장 기본값으로 안정적인 수렴을 기대할 수 있다. Batch size는 64로 설정하여 계산 효율성과 일반화 성능 간의 균형을 고려하였고, Epoch 수는 50으로 초기 학습 성능을 탐색하고 향후 개선 방향을 파악하기 위한 예비적 설정이다.

- 학습 구조

각 epoch 마다 train loss와 validation loss를 추적하여 과적합 여부 실시간 확인하였다. model.eval() 및 torch.no_grad()를 사용하여 평가 시 gradient 계산을 방지했다.

- 성능 평가

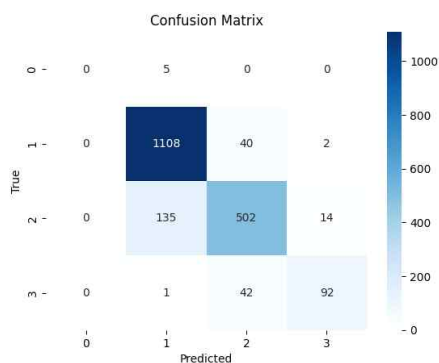
성능 평가는 테스트셋에서 모델이 예측한 결과를 바탕으로 classification_report()와 confusion_matrix()를 통해 수행하였다. 각 클래스별 precision, recall, f1-score 등의 주요 지표를 확인하고, confusion matrix를 시각화함으로써 클래스 간의 혼동 양상도 함께 분석했다.



loss curve 분석

-train loss와 validation loss가 동일한 추세로 안정적 감소, 과적합 없고 학습이 잘 수렴됨.

[그림4. 초기 loss curve 그래프]



Classification Report:

	precision	recall	f1-score	support
0	0.0000	0.0000	0.0000	5
1	0.8871	0.9635	0.9237	1150
2	0.8596	0.7711	0.8130	651
3	0.8519	0.6815	0.7572	135
accuracy			0.8769	1941
macro avg	0.6496	0.6040	0.6235	1941
weighted avg	0.8731	0.8769	0.8726	1941

[그림5. 초기 confusion matrix]

confusion matrix 분석

- class1 : 아주 정확히 잘 맞춤(1108/1150)
- class2 : 절반 이상 맞췄지만 class1이랑 혼동하는 경우 많음
- class3 : class2와 혼동 많음 일부 1과도 혼동
- class0 : 5건 전부 오분류

초기 모델의 첫 번째 성능 결과에서는 전반적으로 양호한 분류 성능을 보였으며, 특히 loss curve 분석 결과, train loss와 validation loss가 유사한 추세로 안정적으로 감소하는 양상을 나타냈다. 이는 과적합 없이 모델이 학습 데이터와 검증 데이터 모두에서 잘 수렴하고 있음을 의미한다.

(4) 초기 모델링 문제 원인 분석과 해결

Class 0(치매 저위험군)의 경우, 전체 데이터 내 샘플 수가 매우 적고, 사용된 feature들의 분포가 다른 클래스들과 유사하여 모델이 해당 클래스를 구분할 수 있는 명확한 분류 근거를 찾기 어려웠다. 이로 인해 예측 정확도가 크게 떨어졌으며, 대부분 다른 클래스로 오분류 되는 현상이 발생하였다.

또한 class 2와 class 3 사이에서는 혼동이 자주 나타났는데, 이는 실제 데이터에서도 두 위험 단계 간의 경계가 뚜렷하게 구분되지 않고 점진적으로 형성되기 때문이다. 이러한 특성상 feature 기반으로 명확한 컷오프(cut-off)를 설정하기 어려워 모델이 경계선상의 데이터를 올바르게 분류하는데 어려움을 겪는 것으로 분석된다.

이를 해결하기 위해 아래의 방법을 사용하였다.

- drop 추가 등의 모델구조 복잡도 상승, 에폭, 배치 수정
- drop 추가 등의 모델구조 복잡도 상승

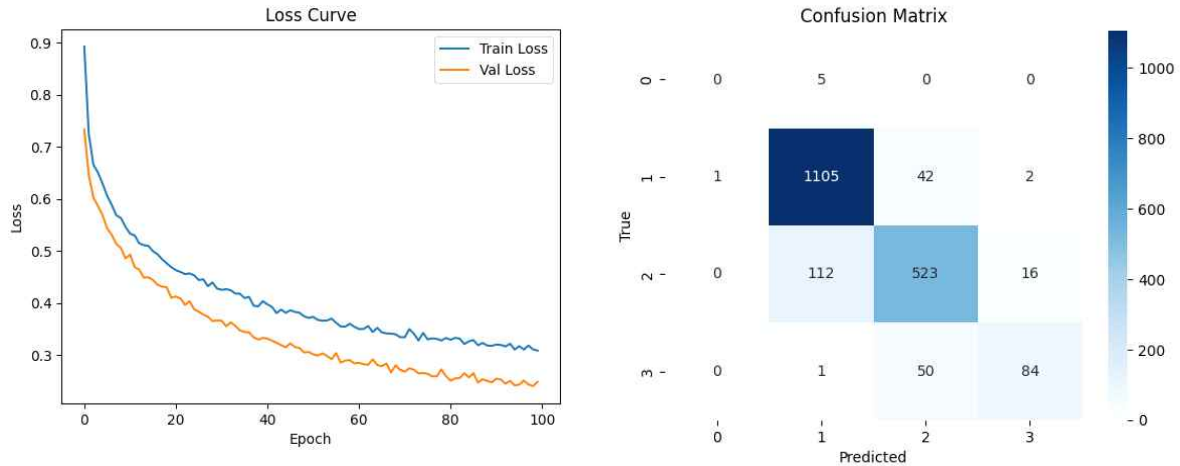
```
self.net = nn.Sequential(  
    nn.Linear(8, 256),  
    nn.ReLU(),  
    nn.Dropout(0.4),  
    nn.Linear(256, 128),  
    nn.ReLU(),  
    nn.Dropout(0.3),  
    nn.Linear(128, 64),  
    nn.ReLU(),  
    nn.Linear(64, 4),  
)
```

모델 구조를 보다 깊게 설계하여 은닉층을 추가하고, Dropout 기법을 도입하여 학습을 진행하였다. 은닉층의 확장은 모델의 표현력을 높여 복잡한 패턴을 더 효과적으로 학습할 수 있도록 하였으며, Dropout은 학습 과정에서 불필요한 과적합을 방지하고 모델의 일반화 성능을 향상시키는데 기여하였다. 이를 통해 클래스 간 구분 성능을 개선하고, 특히 경계가 모호한 클래스에 대한 분류 정확도를 높이고자 하였다.

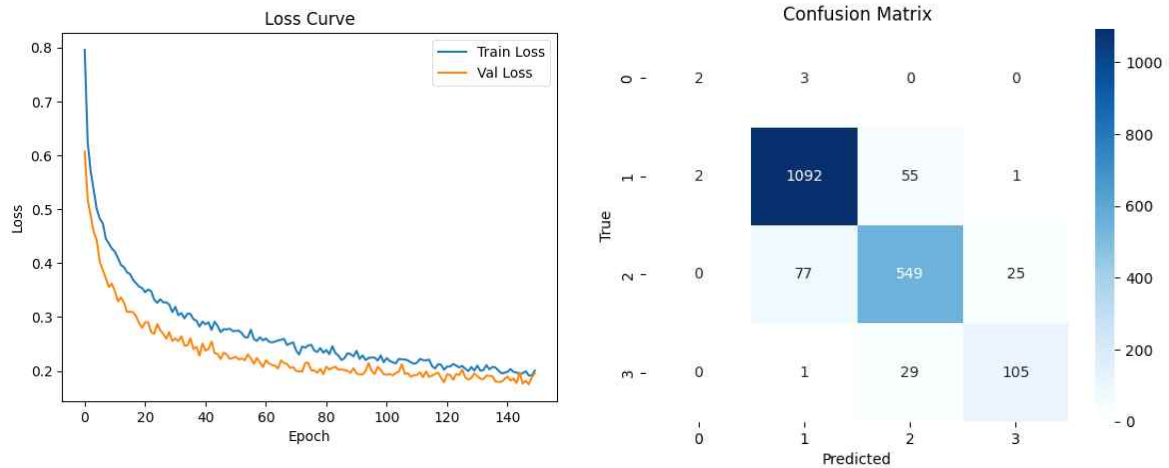
[그림6. 최종 모델 구조]

- 에폭, 배치 수정

[그림 7]은 drop 1개 추가, 에폭 100, 배치 32일 때이고 [그림 8]는 drop 2개 추가, 에폭 150m 배치 32일 때이다. 각각 에폭 50과 100에서 시행했을 때 과적합이 발생하지 않았으나 150으로 실행 했을 때 과적합이 발생하였다. 이 두 결과를 비교해 보니 최종 에폭 120정도가 적합하는 판단을 내렸다.



[그림7. drop 1개 추가, 에폭 100, 배치 32에서의 loss curve, confusion matrix]



[그림8. drop 2개 추가, 에폭 150, 배치 32에서의 loss curve, confusion matrix]

- smote

Class 0(치매 저위험군)은 전체 데이터에서 차지하는 비중이 작아 분류에 어려움이 있었다. 이를 해결하기 위해 SMOTE를 적용하였다. SMOTE는 기존 샘플을 기반으로 소수 클래스 데이터를 인위적으로 생성해 클래스 간 불균형을 해소하고, 모델이 class 0에 대한 학습 기회를 충분히 확보할 수 있도록 도와 예측 성능 향상에 기여하였다.

- 클래스 가중치 계산

소수 클래스에 대한 분류 성능을 향상시키기 위해, 각 클래스의 샘플 수에 반비례하여 클래스별 가중치를 계산하고 이를 PyTorch의 손실 함수에 적용하였다. 학습 중 소수 클래스의 오류에 더 큰 패널티를 부여함으로써 클래스 간 학습 균형을 맞추고, 기존에 성능이 낮았던 class 0에 대해 모델이 보다 민감하게 학습할 수 있도록 설계되었다.

4. 최종 결과물 구현

최종보고서의 '다. 프로젝트 개발 결과' 확인